

Zadanie hackathonowe — Hackology II

1. Tytuł i krótki opis

Zbuduj detektor produktów, który automatycznie rozpoznaje produkty na półce sklepowej.

Celem zadania jest przygotowanie rozwiązania do detekcji produktów na zdjęciach półek sklepowych. Rozwiązanie ma wspierać pracę przedstawicieli handlowych oraz automatyzować procesy wymagające rzetelnej informacji o sytuacji na półce.

2. Kontekst i użytkownik

W procesach sprzedażowych kluczowa jest wiarygodna informacja o tym, jakie produkty znajdują się na półce, w jakiej liczbie i w jakim układzie. Manualna analiza zdjęć jest czasochłonna i trudna do skalowania.

Wyzwania domenowe: gęsto upakowane obiekty, duże podobieństwo wariantów opakowań, niebalansowanie klas, ograniczona liczba danych dla części klas.

3. Cel, metryka i kryteria akceptacji

Cele: skuteczna detekcja produktów (również trudne przypadki), czytelne pokazanie podejścia i kompromisów, rozwiązanie nadające się do dalszego rozwoju.

Metryka: $mAP@0.5$ — średnia precyzja detekcji przy progu IoU ≥ 0.5 , liczona na zbiorze testowym organizatora.

Kryteria akceptacji — zgłoszenie jest poprawne, jeśli:

1. `predictions.json` jest poprawny strukturalnie (format COCO detection),
2. wszystkie `image_id` odpowiadają obrazom ze zbioru public test,
3. wszystkie `category_id` należą do dostarczonej taksonomii,
4. $mAP@0.5 > 0$ (model wykrywa cokolwiek poprawnie),
5. repozytorium zawiera `README.md` z opisem podejścia,
6. `predict.py` działa z CLI: `python predict.py --input <dir> --output predictions.json`.

4. Zakres

W zakresie: detekcja produktów w ramach ustalonej taksonomii, demo i opis rozwiązania, analiza błędów.

Poza zakresem: system produkcyjny end-to-end, panele administracyjne, integracje z zewnętrznymi systemami.

5. Dane

Uczestnicy otrzymują:

- zbiór treningowy (zdjęcia + anotacje COCO),
- zbiór syntetyczny (zdjęcia + anotacje COCO),
- zdjęcia public test (bez anotacji),
- taksonomię produktów (`taxonomy.json`).

Uczestnicy nie otrzymują: anotacji do public test, zdjęć ani anotacji private test.

Można korzystać z zewnętrznych danych, jeśli nie łamie to licencji — wymaga udokumentowania w README.

6. Schemat ewaluacji

Model inspirowany Kaggle:

- **W trakcie hackathonu:** drużyny zgłaszają `predictions.json` na public test → widzą swój `mAP@0.5` na leaderboardze. Limit: 5 zgłoszeń/h, 30 łącznie.
- **Po hackathonie:** organizator uruchamia `predict.py` drużyny na ukrytym private test → wynik końcowy.

Drużyna widzi po zgłoszeniu: status walidacji, `mAP@0.5` na public test, liczbę pozostałych zgłoszeń. Nie widzi: wyników per klasa, private test score.

7. Co oddają zespoły

- repozytorium z kodem (w organizacji hackathonowej na GitHub),
- `predict.py` uruchamialny z CLI (`python predict.py --input <dir> --output predictions.json`),
- `pyproject.toml` + `uv.lock` z zależnościami (reprodukowalność: `uv sync --locked && uv run python predict.py ...`),
- wagi modelu (patrz sekcja 7a),
- `predictions.json` zgłoszony przez system submisji,
- `README.md` z opisem podejścia,
- prezentacja (5–7 slajdów, PDF/PPTX w repo): podejście, wizualizacje wyników, metryki, analiza błędów i ograniczeń.

7a. Wagi modelu

Wagi modelu — do wyboru:

1. **W repozytorium** (zalecane dla plików < 100 MB) — po `git clone` masz wszystko.
2. **Na Hugging Face Hub** (zalecane dla plików > 100 MB) — `predict.py` musi pobierać je automatycznie.
3. **Inny publiczny hosting** — pod warunkiem, że `predict.py` pobiera wagi sam, bez manualnych kroków.

Wymaganie: po `uv sync --locked && uv run python predict.py --input <dir> --output predictions.json` model musi działać bez dodatkowej interwencji. Timeout 30 minut obejmuje pobieranie wag.

Uwaga: linki do wag muszą być stabilne. Jeśli link przestanie działać przed ewaluacją `private test` — `score = 0`.

7b. Środowisko ewaluacji `private test`

Organizator uruchamia `predict.py` na maszynie z:

- Python 3.11
- CUDA 12.x, GPU z 16 GB VRAM
- dostęp do internetu (pobieranie wag dozwolone)
- timeout: 30 minut na cały pipeline (instalacja zależności + download wag + inference)

Jeśli `predict.py` nie uruchomi się w tym środowisku — `score = 0`.

8. Kryteria oceny

Kryterium	Waga	Opis
Jakość detekcji	50%	mAP@0.5 na private test
Jakość techniczna	20%	Czystość kodu, architektura, reprodukowalność
Potencjał wdrożeniowy	15%	Skalowalność, analiza ograniczeń, obsługa edge case'ów
Prezentacja	15%	Czytelność, wizualizacje, wyjaśnienie kompromisów

9. Zasady gry

- dowolne technologie i frameworki, modele pretrenowane, dane dozwolone (jeżeli nie jest to sprzeczne z ich licencją),
- narzędzia AI (Copilot, ChatGPT, Claude) dozwolone do kodowania i jako elementy pipeline'u — wymaga opisu w README,
- zabronione: próby dostępu do ukrytego test setu, publikacja danych konkursowych bez zgody,
- wykorzystanie płatnych API/chmury wymaga opisu w README.

10. Poziomy trudności

Poziom	Opis
1 — Baseline	Działający detektor, poprawny <code>predictions.json</code> , README
2 — Optymalizacja	Augmentacje, fine-tuning, wykorzystanie danych syntetycznych, analiza błędów per klasa
3 — Innowacja	Hierarchia taksonomii, ensembling, few-shot, confidence calibration

11. Starter pack

Uczestnicy otrzymają template repo z:

- danymi (train + synthetic + public test),
- taksonomią,
- skryptami: `validate.py` (walidacja offline), `score.py` (lokalny mAP), `submit.py` (zgłoszenie), `visualize.py`,
- notebookiem z eksploracją danych,
- baseline modelem.